CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in
Approved by | Affiliated to | UGC Autonomous | 14 Programs Accredited by NBA | Grade A++ in NAAC | All India Ranking 151-200 Band nirf | Certifications Quality ISO 21001 : 2018 Green ISO 14001 : 2015 Energy ISO 50001 : 2018
COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

CNN Model Optimization Using a Hybrid Approach of Genetic Algorithm-based Pruning and Retraining with Knowledge Distillation

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Osmania University)

SOFT COMPUTING (22CAE01N)

CASE STUDY REPORT

Submitted By:

160124748093 – Bheeshma Shankar A

CASE STUDY GUIDE

Dr. Y. C. A. Padmanabha Reddy
Associate Professor

Department of CSE Artificial Intelligence and Machine Learning
CBIT, Hyderabad

Academic Year 2026–2027

Certificate

This is to certify that the case study report entitled **"CNN Model Optimization Using a Hybrid Approach of Genetic Algorithm-based Pruning and Retraining with Knowledge Distillation"** has been successfully carried out by the student during the academic year 2026–2027 under my guidance and supervision. The work presented in this report is a record of the student's sincere efforts and academic study.

160124748093 – Bheeshma Shankar A

Case Study Guide:
Dr. Y. C. A. Padmanabha Reddy
Associate Professor
Department of CSE AI&ML
CBIT, Hyderabad

Acknowledgement

I express my sincere gratitude to my case study guide, Dr. Y. C. A. Padmanabha Reddy, Associate Professor, Department of CSE (Artificial Intelligence and Machine Learning), CBIT, for his valuable guidance, continuous support, and encouragement throughout the completion of this Soft Computing case study. His suggestions and explanations helped me understand the concepts of genetic algorithms, structured CNN pruning, and knowledge distillation, and apply soft computing techniques to the selected problem.

I would also like to express my gratitude to Dr. Y. Ramadevi, Head of the Department, Department of CSE (AI & ML), for providing the necessary resources and a supportive academic environment that encouraged me to complete this case study successfully. I am also thankful to Chaitanya Bharathi Institute of Technology (CBIT) for providing the academic facilities and learning opportunities.

Finally, I am grateful to my family and friends for their constant support, encouragement, and motivation throughout the completion of this case study.

A. Bheeshma Shankar - 160124748093

Abstract

Deep convolutional neural networks (CNNs) give strong accuracy, but their size and computation make them hard to run on mobile and embedded devices. This case study analyses the paper by Chou et al. [1], which compresses CNNs by Genetic Algorithm (GA)-guided structured channel pruning followed by retraining with Knowledge Distillation (KD), and reports its main results: for the largest gesture-recognition classifier, 91.02% fewer parameters and 90.96% fewer FLOPs for a 1.82 percentage-point accuracy drop. The report then applies a simplified version of the method to CIFAR-10 in PyTorch on a Tesla T4 GPU. A three-layer baseline CNN with 620,810 parameters reached 79.33% test accuracy. Taylor channel importance and a small GA (population 6, three generations) chose pruning ratios that left 28, 64 and 94 channels. After KD retraining, the student had 459,436 parameters (25.99% fewer), 8.7557 M MACs (19.29% fewer), 79.00% test accuracy (0.33 points lower) and a macro-F1 of 0.7899. Measured single-image latency rose from 1.3488 ms to 1.6645 ms (23.41% slower), so fewer operations did not give faster inference here. The comparisons that would separate the effects of the GA and of KD have not been run.

Keywords: Genetic algorithm, structured channel pruning, knowledge distillation, model compression, soft computing, CIFAR-10.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

List of Figures

Figure	Title	Page No.
2.1	Structured versus unstructured pruning (conceptual illustration).	4
4.1	Overall hybrid framework, from the baseline CNN to the compared compressed model.	7
4.2	Genetic Algorithm workflow for searching the pruning vector.	9
4.3	Teacher–student knowledge distillation used for retraining (RKD).	10
4.4	Computed curves of (a) the initial pruning ratio and (b) the temperature schedule.	11
5.1	Pipeline implemented in the notebook, with the measured result of each stage.	12
5.2	Share of parameters and MACs by layer for the baseline CNN.	13
5.3	Planned five-model comparison: models run and models still to be run.	15
6.1	Reported results for classifiers A to D: (a) accuracy, (b) parameter and FLOP reduction, (c) inference time.	16
6.2	Accuracy drop of LRW (200 epochs) and the proposed RKD (150 epochs).	17
6.3	Training loss and accuracy of the baseline CNN over eight epochs.	17
6.4	Student training loss and validation accuracy during KD retraining, compared with the teacher.	19
6.5	Teacher and pruned student compared on test accuracy, parameters, MACs and latency.	20
6.6	Confusion matrix of the pruned student with KD on the CIFAR-10 test set.	21

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వీన్ భవ
1979

List of Tables

Table	Title	Page No.
2.1	Comparison of the four approaches (the Basis column separates the paper's statements from general comparisons)	3
2.2	Unstructured versus structured pruning	4
2.3	Illustration of GA operators on a three-layer chromosome (hypothetical numbers, not from the paper)	5
3.1	Software and hardware environment of the notebook (Notebook)	6
3.2	Data used in the reference paper and in this study	6
4.1	The ten stages of the hybrid methodology (Paper, Sec. 3)	7
4.2	Symbols and their values (Paper, Sec. 3–4)	8
5.1	Layer-wise parameters and MACs of the baseline CNN (parameters: Notebook; MACs: derived)	12
5.2	Training configuration of the baseline CNN (Notebook)	13
5.3	Genetic Algorithm settings of the reference paper compared with this study	14
5.4	Distillation settings of the reference paper compared with this study	14
6.1	Original versus pruned R(2+1)D-18 classifiers on EgoGesture (Paper, Table 1)	15
6.2	Pruned ResNets compared with two earlier methods (Paper, Table 2)	16
6.3	Retraining with RKD versus LRW [6] (Paper, Table 3)	16
6.4	Training log of the baseline CNN on CIFAR-10 (Notebook)	17
6.5	Best candidate found by the GA after each generation (Notebook)	18
6.6	Layer-by-layer comparison of the teacher and the pruned student (widths and totals: Notebook; per-layer values: derived)	18
6.7	Retraining log of the pruned student with KD (Notebook)	18
6.8	Final comparison of the teacher CNN (M1) and the GA-pruned student with KD (M5) (Notebook)	19
6.9	Per-class precision, recall and F1-score of the pruned student with KD on the test set (Notebook)	20
6.10	Measured results and status of the planned comparison (Notebook)	21
6.11	Advantages and limitations of the hybrid approach	23
7.1	Next steps for completing the experiment	24

Table of Contents

S.No	Title	Page No.
1.	**1. Introduction**	**1**
	1.1 Background and Significance	1
	1.2 The Role of Soft Computing in Model Compression	1
	1.3 Key Challenges in CNN Compression	1
	1.4 Motivation and Report Structure	1
2.	**2. Literature Review and Theoretical Study**	**2**
	2.1 Evolution of Model Compression	2
	2.2 Analysis of the Reference Paper	2
	2.3 Theoretical Background	3
	2.4 Synthesis and Research Gap	5
3.	**3. System Requirements and Specifications**	**5**
	3.1 Hardware and Software Requirements	5
	3.2 Dataset Description and Characteristics	6
	3.3 Evaluation Metrics	6
4.	**4. Methodology and System Design**	**7**
	4.1 Methodology Workflow	7
	4.2 Proposed Hybrid Pipeline: Mathematical Formulation	8
	4.3 Algorithm	11
5.	**5. Implementation Details and Experimental Setup**	**12**
	5.1 Implementation Environment	12
	5.2 Baseline CNN (Teacher) Architecture	12
	5.3 Training Protocol	13
	5.4 Taylor Channel-Importance Scores	13
	5.5 Genetic Algorithm Implementation	13
	5.6 Physical Pruning and KD Retraining	14
	5.7 Experimental Design and Status	14
6.	**6. Results and Performance Analysis**	**15**
	6.1 Overview	15
	6.2 Results Reported in the Reference Paper	15
	6.3 Results from the Experiments of this Study	17
	6.4 Theoretical Interpretation of Results	21
	6.5 Critical Evaluation	22
	6.6 Applications, Advantages and Limitations	23
	6.7 Summary of Findings	23
7.	**7. Conclusion and Future Scope**	**24**
	7.1 Conclusion	24
	7.2 Limitations	24
	7.3 Roadmap and Future Scope	24
8.	**8. Appendix I: Notebook Summary, Glossary and Verification**	**25**
	8.1 Notebook Cells and Status	25
	8.2 Glossary	25
	8.3 Numerical Verification Log	26
9.	**9. References**	**27**

BHARATHI INSTITUTE OF TECHNOLOGY 1979

1. Introduction

1.1 Background and Significance

A Convolutional Neural Network (CNN) learns visual patterns with small shared filters that slide over an image or a video frame. Early layers pick up edges and textures, and deeper layers combine them into parts and whole objects [7]. CNNs now power image recognition, object detection and gesture recognition.

This accuracy has a price. Every convolutional layer stores one filter for each output channel, and a deep network stacks many layers, so the parameter count quickly reaches millions. The largest classifier in the reference paper has 31.34 M parameters and needs 40.52 G floating-point operations (|FLOP|) [1]. AR/VR headsets and mobile phones have limited memory, battery and time budgets, so the authors argue that model size and computation must come down before such models can run in real time on the device [1].

The reference paper studied in this case study, by Chou et al. [1], proposes a hybrid compression method. A Genetic Algorithm (GA) chooses how many channels to prune in each layer, the channels are removed, and the pruned network is retrained with Knowledge Distillation (KD). This report analyses that paper, restates its mathematics, reviews its reported results, and documents the student's own implementation of a simplified version of the pipeline on CIFAR-10, together with the measured results.

1.2 The Role of Soft Computing in Model Compression

Soft Computing covers methods that find good, not necessarily perfect, answers to hard problems, such as evolutionary algorithms, neural networks and fuzzy systems. A Genetic Algorithm is an evolutionary method: it keeps a population of candidate solutions and improves them through selection, crossover and mutation, without gradients and without trying every option. Choosing a pruning ratio for each layer is a search over a very large space; even ten possible ratios per layer give 10^N combinations for N layers (*analysis*). The quality of a configuration (accuracy, size, FLOPs) is not a smooth, differentiable function of the ratios (*analysis*), which makes a heuristic population-based search a natural fit.

1.3 Key Challenges in CNN Compression

- **Search space:** the number of possible per-layer pruning configurations is huge, so manual tuning is slow and trial-and-error.
- **Accuracy loss:** removing channels damages the network, and the damage must be repaired by retraining.
- **Retraining cost:** label-only retraining methods such as fine-tuning and learning-rate rewinding need many epochs [6].
- **Real speed-up:** fewer parameters do not always mean faster inference. In the paper, unstructured pruning removed 80% of the parameters but gave almost no CPU speed-up on a HoloLens [1].
- **Reproducibility:** the paper leaves several settings unspecified (population size, selection and mutation rates, training details), which makes exact reproduction difficult.

1.4 Motivation and Report Structure

The motivation of this case study is to understand, check and begin to reproduce a method that shrinks a CNN by about 90% without a large loss in accuracy. To keep sources clear, the report uses four labels: **(Paper)** for settings and results reported in [1]; **(Notebook)** for numbers printed by the student's own Colab notebook; *analysis* for interpretation or values derived from those numbers; and **Proposed** for work that has not been run. Where [1] gives no detail, the report says "Not specified in the reference paper."

Chapter 2 reviews the literature and the theory, and analyses the reference paper. Chapter 3 lists the requirements, data and evaluation metrics. Chapter 4 describes the methodology and its mathematics. Chapter 5 gives the implementation details and the experimental setup, including the baseline CNN. Chapter 6 presents and interprets the results. Chapter 7 concludes with limitations and a roadmap, Chapter 8 (Appendix I) collects supporting material, and Chapter 9 lists the references.

2. Literature Review and Theoretical Study

2.1 Evolution of Model Compression

2.1.1 Unstructured and Structured Pruning

Pruning removes parts of a trained network that contribute little. Unstructured pruning sets individual small weights to zero [3]. It reduces the number of non-zero parameters, but the zeros sit at arbitrary positions, so special hardware or libraries are needed to turn them into real savings. Structured pruning removes whole filters or channels [4] and keeps the network regular, which is why it is preferred in practice. The importance of a channel can be scored with L1 or L2 norms, or with a Taylor expansion of the loss [5], which the reference paper uses [1].

2.1.2 Retraining after Pruning

Pruning lowers accuracy, so the network is retrained. Fine-Tuning (FT) retrains with a small learning rate, and Learning Rate Rewinding (LRW) replays part of the original learning-rate schedule and is reported to recover accuracy better [6]. Both minimise only the loss against the labels.

2.1.3 Knowledge Distillation

Knowledge Distillation lets a large teacher network guide a smaller student by passing on its softened output probabilities, not only the hard labels [2]. This gives the student extra information about how classes relate to each other, and it is the idea the reference paper adds to the retraining step.

2.2 Analysis of the Reference Paper

The reference paper is by K.-L. Chou, C.-L. Wang, Y.-C. Chen, W. Tang, Y.-T. Li, S.-C. Chang and C.-Y. Wang, published at the Great Lakes Symposium on VLSI (GLSVLSI ’25) [1]. It asks how to reduce a CNN’s size and computation for resource-constrained devices while keeping its accuracy. The authors contrast unstructured and structured pruning, show with a HoloLens measurement that unstructured pruning gave almost no CPU speed-up (Paper, Fig. 1), and therefore choose structured channel pruning. They use a GA because it can explore many candidates and balance several goals at once (Paper). A usual GA string over millions of weights would be too long, so the chromosome holds one pruning ratio per layer instead.

For retraining, the authors argue that FT and LRW ignore the original model’s outputs and treat all wrong classes alike. Their method, RKD, uses the original model as teacher and the pruned model as student (Paper). The paper does not discuss manual per-layer tuning in detail; its stated open challenge is finding a good balance between model size and accuracy. The main contributions are:

- A compact chromosome of layer-wise pruning ratios, with a starting ratio that grows with the number of channels plus noise in [−0.2, 0.2] (Paper).
- Two crossover operators (uniform and arithmetic) applied to two halves of the population, with mutation and top-half selection (Paper).
- A fitness function that combines accuracy, parameter count and FLOPs, with $\alpha = 1$, $\beta = 4$ and $\gamma = 4$ (Paper).
- RKD: a weighted sum of a label loss and a KL-divergence loss, with a temperature falling linearly from 20 to 1 (Paper).

- Evaluation on a video gesture model and on ResNet-32 and ResNet-56 (Paper).

Table 2.1: Comparison of the four approaches (the Basis column separates the paper's statements from general comparisons)

Approach	Core idea	Strengths	Limitations	Basis
Conventional pruning	Remove weights (unstructured) or whole filters/channels (structured) with low importance [3], [4], [5]	Fewer parameters; structured form keeps dense tensors and suits standard hardware	Per-layer ratios must be chosen; accuracy drops; unstructured form gave almost no CPU speed-up [1]	Paper [1] Sec. 1–2; general
GA-based pruning	Encode per-layer ratios as a chromosome and evolve them by crossover, mutation and selection	Searches many configurations without gradients; can weigh accuracy, size and FLOPs together	Needs many fitness evaluations; extra design choices (population, rates, weights)	Paper Sec. 2.3, 3.1; cost point is *analysis*
Knowledge Distillation	A student learns from the teacher's softened outputs as well as from labels [2]	Passes on class-similarity information; can speed up recovery	Needs a teacher; adds temperature and loss-weight settings	Paper Sec. 2.4, 3.2; [2]
Combined framework	GA finds P_{best}; channels are pruned; the student is retrained with KD	Reported >90% fewer parameters and FLOPs on the gesture model; RKD matched LRW-level accuracy in 150 vs 200 epochs	Only three networks tested; no ablation separating GA from KD (*analysis*)	Paper Sec. 3–4; limits are *analysis*

2.3 Theoretical Background

2.3.1 Convolutional Neural Networks

A CNN is built from a few repeated parts. A **convolutional layer** slides filters over its input and produces feature maps; each filter creates one output channel. An **activation function**, commonly ReLU, f(x) = max(0, x), adds non-linearity. **Pooling** shrinks the feature maps. **Fully connected layers** with a softmax turn the features into class probabilities. In this report a *channel* is one feature map and a *filter* is the set of weights that produces it, so removing a filter removes its output channel. For a convolutional layer with C_{in} input channels, C_{out} filters and a $K_h \times K_w$ kernel, the number of weights (bias ignored) and the multiply–accumulate operations (MACs) for an $H_{out} \times W_{out}$ output map are:

$$P_{conv} = C_{out} \cdot C_{in} \cdot K_h \cdot K_w \quad (1)$$

$$MACs_{conv} = H_{out} \cdot W_{out} \cdot C_{out} \cdot C_{in} \cdot K_h \cdot K_w \quad (2)$$

Example (general): a layer with $C_{in} = 64$, $C_{out} = 128$, a 3×3 kernel and a 32×32 output map has $128 \times 64 \times 9 = 73{,}728$ weights and $32 \times 32 \times 128 \times 64 \times 9 = 75{,}497{,}472$ MACs. If a fraction p_l of the output channels of layer l and a fraction p_{l-1} of those of the layer before it are removed, the layer keeps $(1 - p_l)(1 - p_{l-1})$ of its weights (*analysis*):

$$P' = (1 - p_l)(1 - p_{l-1})P_{conv} \quad (3)$$

Pruning 70% of the channels in every layer therefore leaves $0.3 \times 0.3 = 9\%$ of the weights, a 91% reduction. This only illustrates why a roughly 90% parameter cut needs about 70% of channels removed; the paper does not report its per-layer ratios.

2.3.2 Structured Channel Pruning

Structured channel pruning removes entire channels together with their filters, whereas unstructured pruning zeroes single weights (Figure 2.1 and Table 2.2). The benefit is real speed-up without special hardware. The trade-off is that removing a whole channel is a bigger change than zeroing one weight, so accuracy can fall more and retraining is needed. The paper scores channel importance with a Taylor

expansion [5]; the formula is not reproduced in [1], but the idea is to estimate how much the loss would change if the channel were removed.

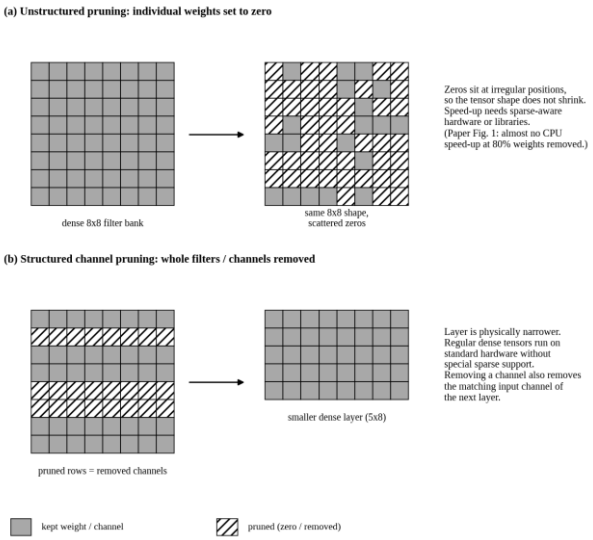


Figure 2.1: Structured versus unstructured pruning (conceptual illustration). *Source: original diagram by the author; the HoloLens observation is from Fig. 1 of [1].*

Table 2.2: Unstructured versus structured pruning

Aspect	Unstructured (weight) pruning	Structured (channel) pruning
What is removed	Individual weights, at arbitrary positions	Whole filters / channels (and the matching input channels of the next layer)
Tensor shape	Unchanged; the tensor becomes sparse	Smaller dense tensors
Hardware need	Sparse-aware support to gain speed	Runs on standard hardware
Latency effect	Often small (paper Fig. 1: almost no CPU speed-up at 80% removal) (Paper)	Real reduction (paper Table 1: 40.63–54.17%) (Paper)
Accuracy risk	Finer granularity, usually gentler (*general*)	Coarser, so a higher risk of accuracy loss (*general*)

2.3.3 Genetic Algorithm

A Genetic Algorithm imitates natural selection. Its parts, as used for pruning, are:

- **Population and chromosome:** a chromosome is one candidate solution, here a vector with one pruning ratio (gene) per layer. The population is the set of M such vectors.
- **Initial population:** M different vectors, built from a size-aware starting ratio plus random noise.
- **Fitness evaluation:** a score for each vector; a higher score means a better balance of accuracy, size and computation.
- **Selection:** the better individuals are kept for the next generation (the paper keeps the top half).
- **Crossover:** two parents are mixed into an offspring that inherits genes from both.
- **Mutation:** a small random change to some genes, which keeps diversity and avoids local optima.
- **New generation and stopping:** the steps repeat until a predefined number of generations (10 to 20 in the paper).

Example (Paper, Example 1): a model with three layers has ChannelCounts = [128, 64, 256]. The chromosome [0.2, 0.32, 0.55] means 20%, 32% and 55% of the channels are pruned in layers 1, 2 and 3, which is 25.6, 20.48 and 140.8 channels, so a rounding rule is needed; the paper does not state one. Table 2.3 shows how the operators act on such vectors, with illustrative numbers chosen by the author.

Table 2.3: Illustration of GA operators on a three-layer chromosome (hypothetical numbers, not from the paper)

Operation	Setting	Result
Parents	P_i = [0.20, 0.32, 0.55]; P_j = [0.40, 0.10, 0.35]	–
Uniform crossover	selection_rate = 0.5; $rand_C$ = (0.3, 0.8, 0.1)	[0.20, 0.10, 0.55]
Arithmetic crossover	r = 0.7	[0.26, 0.254, 0.49]
Mutation	gene 2 mutates; mutation_factor = +0.05	[0.26, 0.304, 0.49]

2.3.4 Knowledge Distillation

A normal classifier trained on hard labels learns only "this is class 3". A teacher's output probabilities also say "and it looks a little like class 7". These **soft targets** carry extra information, and the softmax with **temperature** T makes them softer [2]:

$$q_k = \frac{exp(z_k / T)}{\sum_j exp(z_j / T)} \tag{4}$$

Here z_k is the logit (raw output) for class k. A higher T gives a flatter distribution. *Example (general):* for logits z = [3, 1, 0.2], T = 1 gives probabilities [0.836, 0.113, 0.051], while T = 4 gives [0.475, 0.288, 0.236]. Training combines two standard losses, cross-entropy against the true labels y and KL divergence between the softened teacher and student outputs [10]:

$$CE = -\sum_k y_k \, log \, \hat{y}_k \tag{5}$$

$$KL(P \parallel Q) = \sum_k P_k \, log \left(\frac{P_k}{Q_k}\right) \tag{6}$$

Equations (4)–(6) are standard background. The equations that the paper itself reports are given in Chapter 4 and are marked as such.

2.4 Synthesis and Research Gap

The literature offers strong pruning criteria, retraining methods and distillation. What remains open is how to combine them so that layer-wise ratios are found automatically and accuracy is recovered quickly. The reference paper answers this with a GA plus RKD, but it leaves the separate contribution of each part untested and omits several settings. This case study therefore (i) restates the method and checks its numbers against the paper, (ii) implements a simplified, reproducible version on CIFAR-10 and reports measured results, and (iii) identifies the comparisons still needed to isolate the effect of the GA and of distillation.

3. System Requirements and Specifications

3.1 Hardware and Software Requirements

3.1.1 Environment of the Reference Paper

The authors used Python with the PyTorch library on Windows 10 with an Intel i5-13600 CPU, an NVIDIA RTX 4090 GPU and 64 GB of RAM (Paper). Inference timings in Fig. 1 of [1] were taken on a HoloLens CPU; the device for the timings in Table 1 is not specified in the reference paper.

3.1.2 Environment of this Study

The student's experiment runs in a Google Colab notebook on a GPU runtime. The versions below are those printed by the notebook (Notebook). The package upgrade cell also printed dependency warnings: pandas 3.1.0 was installed although google-colab requires pandas 2.2.3 and the cudf packages require pandas below 2.4. These did not stop the experiment, but they should be fixed or pinned for a clean reproduction.

Table 3.1: Software and hardware environment of the notebook (Notebook)

Component	Version / detail	Role
Runtime	Google Colab, Python 3.13	Execution environment
GPU	Tesla T4, 15.64 GB memory	Training
PyTorch	2.14.1 (CUDA 13.0 build); CUDA available: True	Model and training
torchvision	0.29.1	CIFAR-10 loading and transforms
scikit-learn	1.9.1	Macro-F1 (planned)
matplotlib, pandas	3.11.2, 3.1.0	Plots and tables

3.2 Dataset Description and Characteristics

The notebook uses **CIFAR-10** [14]: 60,000 colour images of 32 × 32 pixels in ten classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). The 50,000 training images are split with a fixed random seed (42) into 45,000 for training and 5,000 for validation, and the 10,000 official test images are kept for final evaluation (Notebook). Images are converted to tensors and normalised with the per-channel means (0.4914, 0.4822, 0.4465) and standard deviations (0.2470, 0.2435, 0.2616); no data augmentation is applied. Batch size is 128 with two loader workers.

Table 3.2: Data used in the reference paper and in this study

Item	Reference paper [1] (Paper)	This study (Notebook)	Notes
Gesture data	EgoGesture, 83 hand gestures [13]	Not used	Video data
Image data	CIFAR-10 and CIFAR-100 [14]	CIFAR-10 only	10 classes, 32 × 32
Models	R(2+1)D-18 (classifiers A–D); ResNet-32, ResNet-56	Small 3-conv CNN (Chapter 5)	Baseline / teacher
Splits	Not specified in the reference paper	45,000 / 5,000 / 10,000	Seed 42 for the split

3.3 Evaluation Metrics

- **Accuracy, macro-precision, macro-recall and macro-F1:** top-1 accuracy, and the unweighted mean over the ten classes of precision, recall and F1, measured on the 10,000 CIFAR-10 test images with scikit-learn.
- **Parameters (|Param|) and model size:** the count of learnable parameters and the size of the saved weights.
- **Computation (|FLOP|):** floating-point operations; this report counts multiply–accumulate operations (MACs) and states the convention, since [1] does not state its own.
- **Inference time:** latency in milliseconds per image, measured for batch size 1 on random 3 × 32 × 32 input after 50 warm-up runs and averaged over 300 timed runs, with float16 autocast on the Tesla T4 and torch.cuda.synchronize() around the timing (Notebook).

Reductions are reported as percentage reductions, and accuracy changes as percentage-point differences; the difference between the two is explained with a worked example in Section 6.4.1.

4. Methodology and System Design

4.1 Methodology Workflow

The framework of [1] runs in ten stages, summarised in Table 4.1 and drawn in Figure 4.1. Stages 2 to 6 are the GA search, stages 7 and 8 compress and recover the model, and stages 9 and 10 evaluate it.

Table 4.1: The ten stages of the hybrid methodology (Paper, Sec. 3)

#	Stage	Purpose	Expected output
1	Select original CNN	Provide the trained baseline whose accuracy must be kept; it also serves as the teacher	Baseline weights and accuracy
2	Encode pruning ratios	Represent one pruning ratio per layer as a chromosome	Vector of length N
3	Initialise population	Create M diverse candidates: size-aware starting ratio plus noise	Init = $[P_1, \ldots, P_M]$
4	Evaluate fitness	Score each candidate on accuracy, |Param| and |FLOP|	Fitness per candidate
5	Apply GA operators	Crossover (two groups), mutation and top-half selection, repeated for the set number of generations	Evolved population
6	Select P_{best}	Take the best individual found	Best pruning vector
7	Prune channels	Remove channels in each layer as P_{best} says, least important first (Taylor score)	Pruned (student) network
8	Retrain with KD	Recover accuracy using the label loss plus the teacher's soft targets	Retrained student
9	Evaluate	Measure accuracy, |Param|, |FLOP| and inference time	Metric values
10	Compare	Compare with the baseline and with other methods	Reduction and accuracy change

How the accuracy term is obtained for each GA candidate (for example, whether a candidate is briefly retrained before scoring) is **not specified in the reference paper**. This matters for cost and is discussed in Section 6.5.

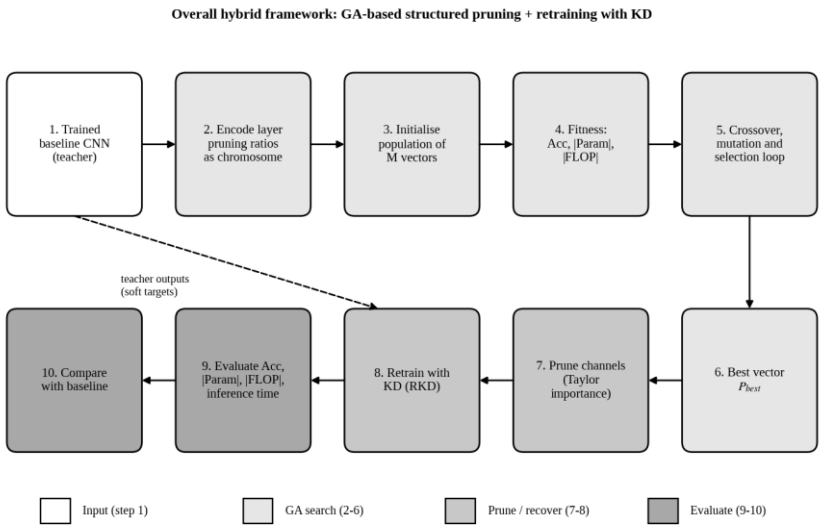


Figure 4.1: Overall hybrid framework, from the baseline CNN to the compared compressed model.
Source: original diagram by the author, based on Sec. 3 and Fig. 4 of [1].

4.2 Proposed Hybrid Pipeline: Mathematical Formulation

Every equation in Sections 4.2.1 to 4.2.5 is taken from the reference paper [1]; the report's equation number is followed by the paper's EQ number. Table 4.2 defines the symbols.

Table 4.2: Symbols and their values (Paper, Sec. 3–4)

Symbol	Meaning	Value in paper
N, $channel_j$	Number of layers; number of channels in layer j	Model-dependent
P_i, $pruned_{i,j}$	Pruning vector of individual i; pruning ratio of layer j	Evolved by the GA
$NF_{i,j}$	Noise factor, uniform random	Range [−0.2, 0.2]
κ, λ	User parameters of the initial pruning ratio	κ = 150, λ = 0.2
M	Population size	Not specified in the reference paper
Generations	Number of GA generations	10 to 20
selection_rate, $rand_C$	Threshold and random draw for uniform crossover	Rate: not specified
r	Random weight of arithmetic crossover	Distribution not specified
mutation_rate, mutation_factor, $rand_M$	Probability, size of change and random draw for mutation	Rate and factor: not specified
Acc, |Param|, |FLOP|	Accuracy; parameter count; floating-point operations	Measured per model
α, β, γ	Fitness weights for accuracy, size and operations	α = 1, β = 4, γ = 4
z_S, z_T, T	Student logits, teacher logits, temperature	T starts at T_0 = 20
δ, E	Weight of the label loss; total retraining epochs	δ = 0.5, E = 150

4.2.1 Chromosome and Initialisation

The channel counts of a model with N layers and the pruning vector of the i-th individual are (paper EQ 1 and EQ 2):

$$ChannelCounts = [channel_1, channel_2, \dots, channel_N] \tag{7}$$

$$P_i = [pruned_{i,1}, pruned_{i,2}, \dots, pruned_{i,j}, \dots, pruned_{i,N}] \tag{8}$$

Layers with more channels can tolerate more pruning, so the starting ratio grows with the channel count (paper EQ 4). Noise gives each individual a different value (paper EQ 3 and EQ 5), and the population collects M such vectors (paper EQ 6):

$$Initial_Pruning_Ratio_j = \left(1 - e^{-\lambda \times \frac{channel_j}{\kappa}}\right) \tag{9}$$

$$pruned_{i,j} = \left(1 - e^{-\lambda \times \frac{channel_j}{\kappa}}\right) + NF_{i,j} \tag{10}$$

$$Init = [P_1, P_2, \dots, P_M] \tag{11}$$

With κ = 150 and λ = 0.2, Equation (9) gives starting ratios of 0.082, 0.157, 0.289 and 0.495 for layers of 64, 128, 256 and 512 channels (computed from the paper's equation; Figure 4.4a). Equation (10) then adds noise between −0.2 and +0.2. How a ratio that falls outside [0, 1] after noise is handled is not specified.

4.2.2 Crossover and Mutation

The population is split into two groups: one uses uniform crossover [8] and the other arithmetic crossover [9]. In uniform crossover each gene comes from parent P_i if a random number $rand_C$ is below

selection_rate, and otherwise from parent P_j (paper EQ 7). In arithmetic crossover a random weight r blends the two parents (paper EQ 8):

$$pruned_{i,j}^{Offspring} = pruned_{i,j}^{Pi},\ if\ rand_C < selection_rate \tag{12}$$

$$pruned_{i,j}^{Offspring} = pruned_{i,j}^{Pj},\ otherwise$$

$$pruned_{i,j}^{Offspring} = r \cdot pruned_{i,j}^{Pi} + (1 - r) \cdot pruned_{i,j}^{Pj} \tag{13}$$

For mutation, each gene draws $rand_M$. If it is below mutation_rate, the gene changes by a constant mutation_factor (paper EQ 9):

$$pruned'_{i,j} = pruned_{i,j} + mutation_factor,\ if\ rand_M < mutation_rate \tag{14}$$

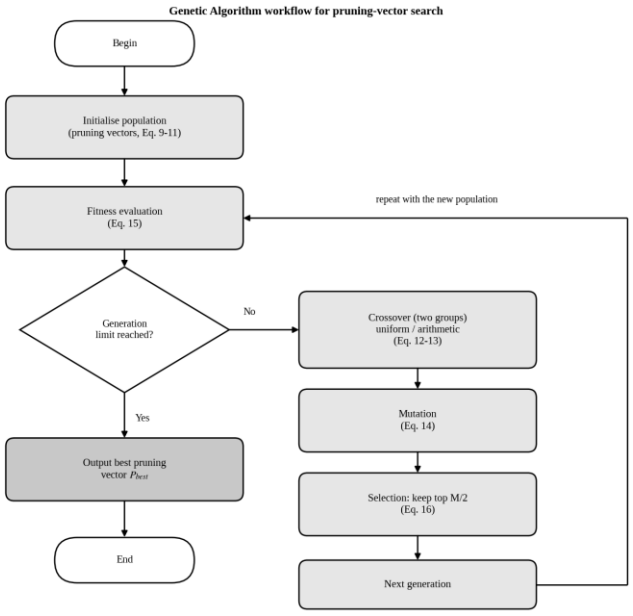


Figure 4.2: Genetic Algorithm workflow for searching the pruning vector. *Source: original diagram by the author, based on Sec. 3.1 and Fig. 2 of [1].*

INSTITUTE OF TECHNOLOGY

4.2.3 Fitness Function and Selection

The fitness value balances accuracy, model size and computation (paper EQ 10):

$$Fitness = \alpha \cdot Acc + \frac{\beta}{log(|Param|)} + \frac{\gamma}{log(|FLOP|)} \tag{15}$$

Here Acc is the model's accuracy, |Param| its parameter count and |FLOP| its floating-point operations, with $\alpha = 1$, $\beta = 4$ and $\gamma = 4$ (Paper). The printed equation in [1] writes the third coefficient as "r", while the text defines it as γ; this report reads it as γ (a typographical inconsistency in the source). Because the size and operation terms are inverse logarithms, fewer parameters or FLOPs give a higher fitness, but the reward grows slowly (*analysis*). The log base, the scale of Acc (fraction or percent) and how Acc is measured are not specified in the reference paper. Selection ranks all individuals by fitness and keeps the top half (paper EQ 11):

$$I_{current} = [P_1, \ldots, P_{M/2}] \tag{16}$$

After the final generation, the best individual is called P_{best} and is used to prune the model by channel importance (Paper).

4.2.4 Structured Channel Pruning

Given P_{best}, each layer j loses the fraction $pruned_{best,j}$ of its channels. The channels removed are the least important ones according to a Taylor-expansion score [5] (Paper). Removing an output channel also removes the matching input weights of the next layer, which is why parameters and FLOPs fall faster than the channel count (Equation 3).

4.2.5 Retraining with Knowledge Distillation (RKD)

The student is the pruned model and the teacher is the original model (Figure 4.3). The loss has a label part (paper EQ 12) and a distillation part (paper EQ 13), combined with a weight δ (paper EQ 14):

$$Loss1 = Loss(Softmax(Output_Student), Target\ Label) \tag{17}$$

$$Loss2 = T^2 \times KL_div\left(Softmax\left(\frac{Output_Student}{T}\right), Softmax\left(\frac{Output_Teacher}{T}\right)\right) \tag{18}$$

$$Loss_Total = \delta * Loss1 + (1 - \delta) * Loss2 \tag{19}$$

The paper describes Loss1 as a traditional loss "like cross-entropy", so Equation (5) is its usual form. Loss2 lists the student distribution first and the teacher second; the paper does not spell out the argument convention of KL_div, and common software libraries order these arguments differently. The factor T^2 is the usual scaling that keeps the gradient size comparable when T changes [2] (*general background*). The temperature falls linearly from T_0 to 1 over the E retraining epochs (paper EQ 15), with $T_0 = 20$, E = 150 and δ = 0.5 (Paper); Figure 4.4b plots this schedule:

$$T(epoch) = T_0 - \frac{T_0 - 1}{E} \cdot epoch \tag{20}$$

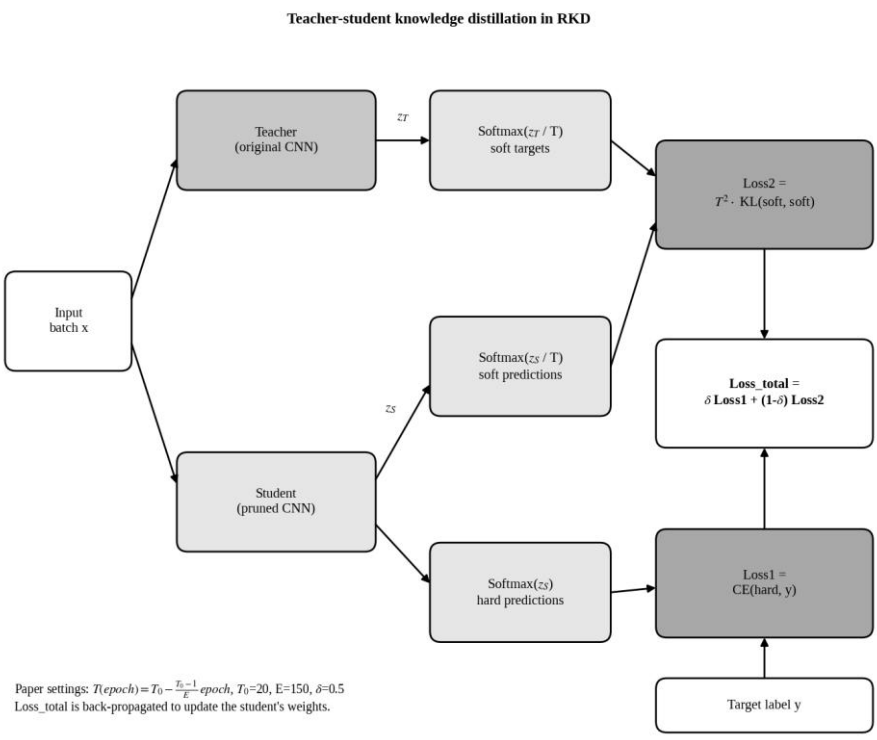


Figure 4.3: Teacher–student knowledge distillation used for retraining (RKD). *Source: original diagram by the author, based on Sec. 3.2 and Fig. 3 of [1].*

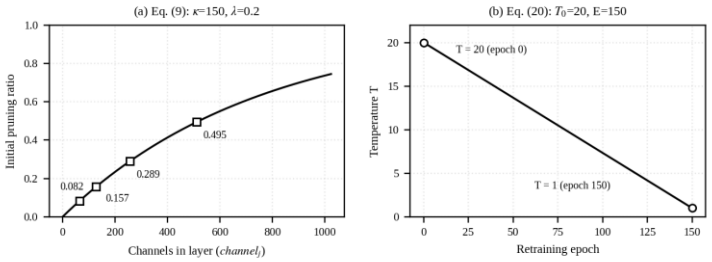


Figure 4.4: Computed curves of (a) the initial pruning ratio and (b) the temperature schedule. *Source: computed by the author from Equations (9) and (20) using κ, λ, T_0 and E reported in [1].*

4.3 Algorithm

Algorithm 4.1 gives the complete hybrid procedure, written by the author from the description in [1].

Line	**Algorithm 4.1: GA-based structured pruning with retraining by KD (after [1])**
1	**Input:** trained CNN F (teacher), training data, M, G, κ, λ, α, β, γ, selection_rate, mutation_rate, mutation_factor, δ, T_0, E
2	**Output:** compressed CNN F_s
3	Read ChannelCounts = [$channel_1$, …, $channel_N$] from F
4	For each individual i = 1..M and layer j = 1..N:
5	$init_j \leftarrow 1 - \exp(-\lambda \cdot channel_j / \kappa)$ (Eq. 9)
6	$pruned[i, j] \leftarrow init_j + U(-0.2, 0.2)$ (Eq. 10)
7	Population $\leftarrow [P_1, \ldots, P_M]$; evaluate the fitness of each individual (Eq. 15)
8	For generation g = 1..G (G = 10 to 20 in the paper):
9	Split the population into two groups
10	Group 1: uniform crossover; group 2: arithmetic crossover (Eq. 12, 13)
11	For each gene: if $rand_M$ < mutation_rate, add mutation_factor (Eq. 14)
12	Evaluate the fitness of all individuals
13	Rank by fitness and keep the top M/2 (Eq. 16)
14	Form the next generation (refilling to size M is not specified in [1])
15	$P_{best} \leftarrow$ best individual found
16	$F_s \leftarrow$ remove channels from F by the ratios in P_{best}, least important first
17	For epoch = 0..E (E = 150):
18	$T \leftarrow T_0 - (T_0 - 1)/E \cdot epoch$ (Eq. 20)
19	For each mini-batch (x, y): $z_T \leftarrow F(x)$; $z_S \leftarrow F_s(x)$
20	$L1 \leftarrow CE(softmax(z_S), y)$ (Eq. 17)
21	$L2 \leftarrow T^2 \cdot KL(softmax(z_S/T), softmax(z_T/T))$ (Eq. 18)
22	$L \leftarrow \delta \cdot L1 + (1 - \delta) \cdot L2$; update the weights of F_s (Eq. 19)
23	Return F_s

Selection needs fitness values and a full-size population is needed in the next round; [1] does not detail either step, so lines 13 and 14 reflect the author's reading.

5. Implementation Details and Experimental Setup

5.1 Implementation Environment

All code runs in the Colab notebook described in Section 3.1.2 using PyTorch [17] on a Tesla T4 GPU. The notebook covers environment checks, data loading, the baseline CNN and its training, Taylor channel importance, a GA search, physical pruning, KD retraining, test-set evaluation, latency and MAC measurement, and result charts. Figure 5.1 shows the stages with the measured result of each; Section 8.1 lists the notebook cells.

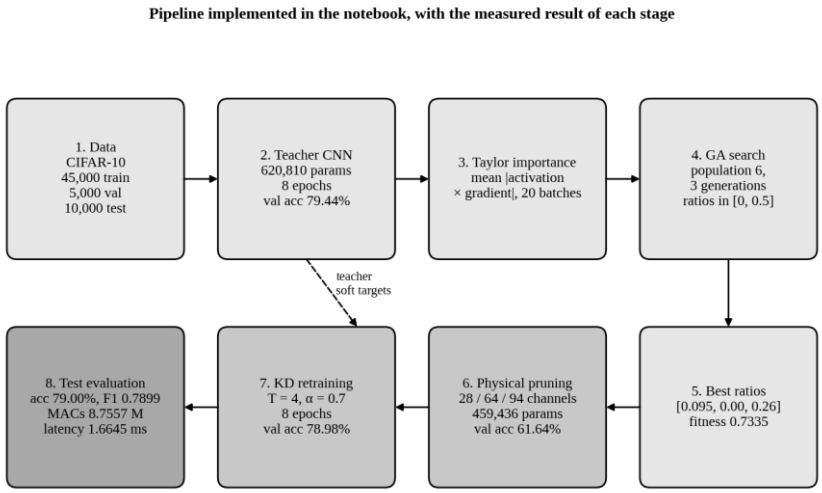


Figure 5.1: Pipeline implemented in the notebook, with the measured result of each stage. *Source: original diagram by the author; numbers from the notebook output.*

5.2 Baseline CNN (Teacher) Architecture

The baseline CNN printed by the notebook has three 3 × 3 convolutional layers (32, 64 and 128 channels), each followed by batch normalisation, a shared 2 × 2 max-pooling module, and two fully connected layers (256 and 10 units). It has **620,810** parameters (Notebook). The notebook's PrunedCNN class applies ReLU and max-pooling after each batch-norm layer and feeds $c_3 \times 4 \times 4$ values to fc1, and the teacher has the same layout (fc1 input $2048 = 128 \times 4 \times 4$), which means three pooling steps on a 32 × 32 input. Table 5.1 gives the parameters from the printed layers and the MACs computed from Equations (1) and (2). The total of 10,848,768 MACs agrees with the 10.8488 million counted by the notebook's own hook-based counter (Notebook).

Table 5.1: Layer-wise parameters and MACs of the baseline CNN (parameters: Notebook; MACs: derived)

Layer	Output (before pooling)	Parameters	MACs	Share of MACs
conv1 (3→32)	32 × 32 × 32	896	884,736	8.16%
bn1 (32)	–	64	–	–
conv2 (32→64)	64 × 16 × 16	18,496	4,718,592	43.49%
bn2 (64)	–	128	–	–
conv3 (64→128)	128 × 8 × 8	73,856	4,718,592	43.49%
bn3 (128)	–	256	–	–
fc1 (2048→256)	256	524,544	524,288	4.83%
fc2 (256→10)	10	2,570	2,560	0.02%
Total	–	**620,810**	**10,848,768**	**100%**

Parameter counts add up to the 620,810 printed by the notebook. MACs ignore biases and batch-norm operations.

The two large convolutions, conv2 and conv3, hold 86.99% of the computation but only 14.9% of the parameters. The first fully connected layer holds 84.49% of the parameters (Figure 5.2). Pruning conv3 channels shrinks fc1 as well, because each conv3 channel feeds 4 × 4 = 16 fc1 inputs, which is 4,096 weights (*analysis*).

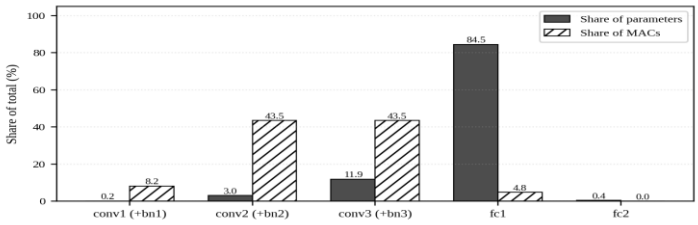


Figure 5.2: Share of parameters and MACs by layer for the baseline CNN. *Source: derived from Table 5.1.*

5.3 Training Protocol

Table 5.2: Training configuration of the baseline CNN (Notebook)

Setting	Value
Loss function	Cross-entropy (nn.CrossEntropyLoss)
Optimiser	AdamW, learning rate 0.001, weight decay 1e-4
Learning-rate schedule	Cosine annealing, T_{max} = 8 epochs, stepped once per epoch
Epochs and batch size	8 epochs; batch size 128
Precision	Mixed precision (float16 autocast with GradScaler)
Model selection	Best validation accuracy checkpoint saved as best_teacher.pth
Data split	45,000 train / 5,000 validation; split seed 42; test set untouched
Augmentation	None (tensor conversion and normalisation only)

5.4 Taylor Channel-Importance Scores

The notebook reloads the best teacher checkpoint and registers forward hooks on conv1, conv2 and conv3 that store each layer's activations and keep their gradients. For 20 training batches it runs a forward and backward pass with the cross-entropy loss and scores each channel by the mean of |activation × gradient| over the batch and spatial positions, then averages the scores over the batches. This is a first-order Taylor-style score of the kind used in [1] and [5]. The printed score ranges (Notebook) are 1.469754e-06 to 1.613475e-05 for the 32 channels of conv1, 2.624851e-06 to 1.205276e-05 for the 64 channels of conv2, and 3.591285e-06 to 3.031291e-05 for the 128 channels of conv3. Channels with the lowest scores are removed first.

5.5 Genetic Algorithm Implementation

The GA in the notebook is a reduced version of the one in [1], sized to run within a Colab session. A candidate is a vector of three pruning ratios, one for each convolutional layer, drawn uniformly from [0, 0.5]. Its fitness is the accuracy of the teacher on a fixed set of 1,024 validation images, with the channels that the Taylor scores rank lowest masked out, plus 0.15 times the fraction of channels removed. There is no retraining inside the search. Each generation keeps the two best candidates as elites, builds new candidates as the mean of two elites, and adds Gaussian noise (standard deviation 0.08) to every new gene before clipping to [0, 0.5]. Table 5.3 compares these choices with the paper.

Table 5.3: Genetic Algorithm settings of the reference paper compared with this study

Setting	Reference paper [1] (Paper)	This study (Notebook)
Chromosome	One pruning ratio per layer	Three genes (conv1, conv2, conv3); fully connected layers are not pruned directly
Population, generations	Population: not specified; 10 to 20 generations	6 individuals, 3 generations
Initialisation	Size-aware starting ratio plus noise, Eq. (9)–(10)	Uniform random in [0, 0.5]
Crossover	Uniform and arithmetic on two groups	Mean of two elite parents (arithmetic with r = 0.5)
Mutation	Constant mutation_factor with a mutation_rate	Gaussian noise, σ = 0.08, on every new gene
Selection	Top half kept	Top two of six kept as elites
Fitness	α · Acc + β / log|Param| + γ / log|FLOP|	Accuracy on 1,024 validation images (masked teacher) + 0.15 × channel reduction
Channel scoring	Taylor expansion	Mean |activation × gradient| over 20 batches

5.6 Physical Pruning and KD Retraining

The best ratios are turned into retained channel counts with keep = max(1, int(original × (1 − ratio))), and the channels with the highest Taylor scores are kept. The notebook then builds a new, smaller network (PrunedCNN) and copies into it the retained convolution weights, the matching input channels of the next convolution, the batch-norm parameters and statistics, and the fc1 columns that belong to the retained conv3 channels (each channel feeds 16 fc1 inputs). The result is a physically smaller model, not a masked one.

The pruned student is then retrained with Knowledge Distillation. The teacher is frozen and put in evaluation mode. The loss is $(1 - \alpha) \cdot$ cross-entropy $+ \alpha \cdot T^2 \cdot$ KL, computed with PyTorch's kl_div on the log-softmax of the student and the softmax of the teacher, so it measures KL(teacher ‖ student). The settings are T = 4, α = 0.7, 8 epochs, AdamW with learning rate 0.001 and weight decay 1e-4, and mixed precision. The checkpoint with the best validation accuracy is saved. Table 5.4 compares this with the paper.

Table 5.4: Distillation settings of the reference paper compared with this study

Setting	Reference paper [1] (Paper)	This study (Notebook)
Temperature	Falls linearly from $T_0 = 20$ to 1	Constant, T = 4
Loss weights	δ = 0.5 on the label loss, 1 − δ on the KL loss	0.3 on the label loss, α = 0.7 on the KL loss
T^2 factor on KL	Yes, Eq. (18)	Yes
Retraining epochs	E = 150	8
Teacher	Original model	Frozen best teacher checkpoint
Optimiser	Not specified in the reference paper	AdamW, learning rate 0.001, weight decay 1e-4

5.7 Experimental Design and Status

The baseline and the GA-pruned KD student are models M1 and M5 of a five-model comparison. Models M2 to M4 (uniform pruning without KD, KD only with a small student, and GA pruning with

label-loss retraining) were not run in the notebook. Figure 5.3 shows which parts were run, and Table 6.10 lists the measured values.

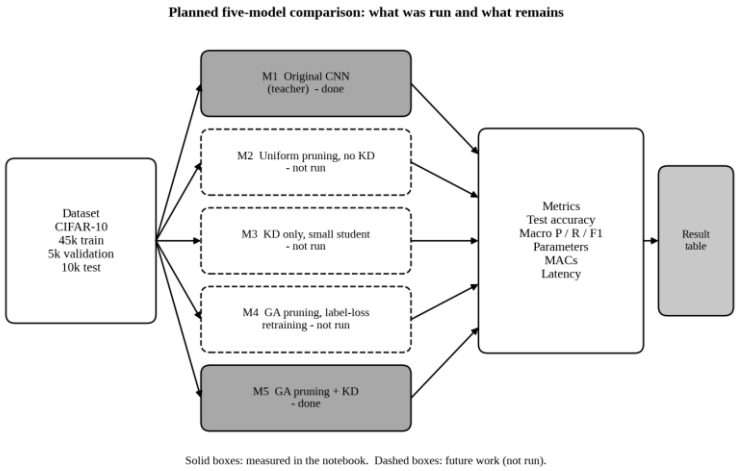


Figure 5.3: Planned five-model comparison: models run and models still to be run. *Source: original design by the author.*

6. Results and Performance Analysis

6.1 Overview

This chapter presents three kinds of results, kept apart: the results reported in the reference paper (Paper), the numbers measured in the student's notebook (Notebook), and values derived from them (*analysis*). The paper's compression results come from different models and datasets than the notebook, so they are not directly comparable. They show what the method can reach at scale, while the notebook tests whether the same ideas work on a much smaller network.

6.2 Results Reported in the Reference Paper

6.2.1 R(2+1)D-18 on EgoGesture

Table 6.1 reproduces Table 1 of [1] for the four classifiers of the self-distilled R(2+1)D-18 model. "Acc drop" is the difference in accuracy in percentage points.

Table 6.1: Original versus pruned R(2+1)D-18 classifiers on EgoGesture (Paper, Table 1)

Clf.	Acc (%) orig → pruned	Acc drop (pp)	|Param| (M) orig → pruned	|Param| ↓ (%)	|FLOP| (G) orig → pruned	|FLOP| ↓ (%)	Time (ms) orig → pruned	Time ↓ (%)
A	85.09 → 77.12	7.97	1.17 → 0.11	90.22	29.28 → 2.66	90.92	2.4 → 1.1	54.17
B	91.34 → 89.01	2.33	2.71 → 0.25	90.71	34.32 → 3.11	90.94	2.8 → 1.4	50.00
C	92.12 → 90.14	1.98	8.73 → 0.79	90.98	38.35 → 3.47	90.96	3.2 → 1.9	40.63
D	**92.24 → 90.42**	**1.82**	**31.34 → 2.81**	**91.02**	**40.52 → 3.66**	**90.96**	**3.6 → 2.1**	**41.67**

Source: [1], Table 1. pp = percentage points; ↓ = reduction relative to the original.

For classifier D, the paper reports accuracy of 92.24% before and 90.42% after pruning, a 1.82 percentage-point drop. Parameters fall from 31.34 M to 2.81 M (91.02% reduction) and FLOPs from 40.52 G to 3.66 G (90.96% reduction). Inference time falls from 3.6 ms to 2.1 ms. All values match Table 1 of [1] and its abstract. Every classifier loses more than 90% of both parameters and FLOPs, and inference time falls by 40.63% to 54.17% (Paper).

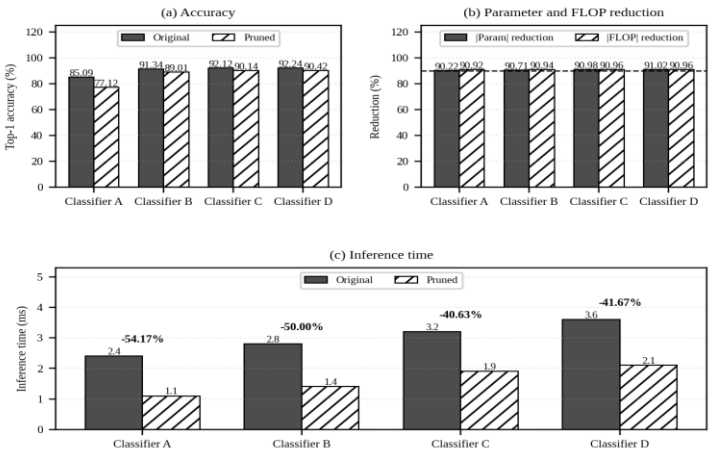


Figure 6.1: Reported results for classifiers A to D: (a) accuracy, (b) parameter and FLOP reduction, (c) inference time. *Source: values from Table 1 of [1].*

6.2.2 ResNet-32 and ResNet-56 on CIFAR-10 and CIFAR-100

Table 6.2: Pruned ResNets compared with two earlier methods (Paper, Table 2)

Dataset	Model	Approach	Ori. Acc (%)	New Acc (%)	Acc drop (pp)	|Param| ↓ (%)	|FLOP| ↓ (%)
CIFAR-10	ResNet-32	Cheng 2023 [16]	92.63	91.02	1.61	–	41.50
		Cheng 2021 [15]	92.63	92.14	0.49	–	53.20
		Ours [1]	92.89	92.17	0.72	50.43	58.14
	ResNet-56	Cheng 2023 [16]	93.59	91.67	1.92	–	54.60
		Ours [1]	93.22	92.83	0.39	63.53	63.85
CIFAR-100	ResNet-32	Cheng 2021 [15]	70.28	69.44	0.84	–	41.50
		Ours [1]	69.94	68.88	1.06	43.06	49.37
	ResNet-56	Cheng 2023 [16]	71.07	70.73	0.34	–	52.60
		Ours [1]	71.04	70.69	0.35	59.93	62.23

Source: [1], Table 2. "–" means not reported. Rows labelled [15] and [16] are cited as [4] and [5] in the paper.

On CIFAR-10, the pruned ResNets keep accuracy within 0.72 and 0.39 points and reduce FLOPs by 58.14% and 63.85%, more than the two earlier methods in the same table. On CIFAR-100, ResNet-32 ends slightly below [15] in accuracy (68.88% against 69.44%) but with a larger FLOP reduction (49.37% against 41.50%). For ResNet-56, the drop is 0.35 points with a 62.23% FLOP reduction, comparable to [16] (Paper). Table 6.3 compares RKD with LRW [6].

Table 6.3: Retraining with RKD versus LRW [6] (Paper, Table 3)

Dataset	Model	Ori. Acc (%)	Approach	New Acc (%)	Acc drop (pp)	Epochs	|Param| / |FLOP| ↓ (%)
CIFAR-10	ResNet-32	92.89	LRW [6] Ours	92.02 92.17	0.87 0.72	200 150	50.43 / 58.14
CIFAR-10	ResNet-56	93.22	LRW [6] Ours	92.88 92.83	0.34 0.39	200 150	63.53 / 63.85
CIFAR-100	ResNet-32	69.94	LRW [6] Ours	68.77 68.88	1.17 1.06	200 150	43.06 / 49.37
CIFAR-100	ResNet-56	71.04	LRW [6] Ours	70.60 70.69	0.44 0.35	200 150	59.93 / 62.23

Source: [1], Table 3.

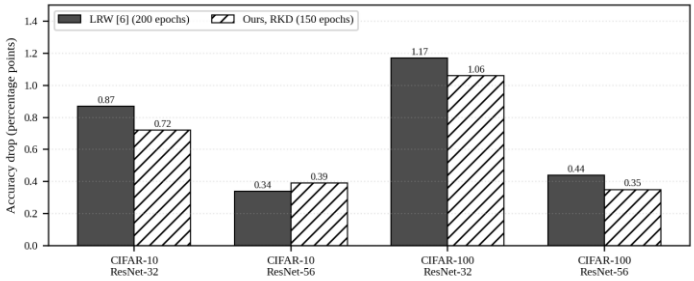


Figure 6.2: Accuracy drop of LRW (200 epochs) and the proposed RKD (150 epochs). *Source: values from Table 3 of [1].*

RKD reaches an accuracy drop similar to LRW in 150 epochs instead of 200, which is 25% fewer epochs. It is slightly better in three of the four settings and slightly worse for ResNet-56 on CIFAR-10 (0.39 against 0.34 points) (Paper).

6.3 Results from the Experiments of this Study

6.3.1 Teacher (Baseline) Training

Table 6.4 lists the per-epoch log printed by the notebook when the baseline CNN was trained for eight epochs on a Tesla T4 GPU (Notebook).

Table 6.4: Training log of the baseline CNN on CIFAR-10 (Notebook)

Epoch	Training loss	Training accuracy (%)	Validation accuracy (%)	Time (s)
1	1.2486	54.95	64.76	13.5
2	0.8455	70.18	68.58	11.6
3	0.6872	75.88	73.98	11.6
4	0.5812	79.68	75.88	12.1
5	0.4793	83.54	76.12	11.5
6	0.3913	86.84	78.04	11.8
7	0.3242	89.44	78.62	11.5
8	**0.2839**	**91.19**	**79.44**	**11.6**

Source: output of the training cell of the student's notebook. Best validation accuracy: 79.44% (epoch 8).

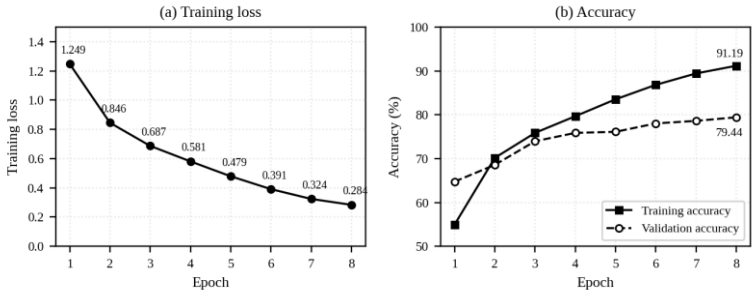


Figure 6.3: Training loss and accuracy of the baseline CNN over eight epochs. *Source: values from Table 6.4 (Notebook).*

Training loss fell steadily from 1.2486 to 0.2839, and validation accuracy rose from 64.76% to 79.44%, which is the best value and the one saved as the teacher checkpoint. Epochs took about 11.9 s on average (95.2 s in total, *analysis* from the log). On the 10,000 test images the same checkpoint scored 79.33% (Section 6.3.4).

6.3.2 GA Search and Pruning Outcome

Table 6.5 shows the best candidate after each generation (Notebook). The best fitness did not change between generation 1 and generation 2 and improved only in generation 3, from 0.7049 to 0.7335, so the search had little time to evolve (*analysis*).

Table 6.5: Best candidate found by the GA after each generation (Notebook)

Generation	Best fitness so far	Best pruning ratios so far (conv1, conv2, conv3)
1	0.7049	0.103, 0.035, 0.295
2	0.7049	0.103, 0.035, 0.295
3	**0.7335**	**0.095, 0.000, 0.260**

The best ratios (0.095, 0, 0.26) give retained widths of 28, 64 and 94 channels. Conv1 loses 4 of its 32 channels, conv2 keeps all 64 because the GA set its ratio to zero, and conv3 loses 34 of its 128. The pruned network has 459,436 parameters, a 25.99% reduction (Notebook). Table 6.6 compares the two networks layer by layer; the pruned parameter and MAC totals recomputed from these widths (459,436 and 8,755,712) match the notebook's printed values (*analysis*).

Table 6.6: Layer-by-layer comparison of the teacher and the pruned student (widths and totals: Notebook; per-layer values: derived)

Layer	Width (teacher → student)	Params (teacher)	Params (student)	MACs (teacher)	MACs (student)
conv1 (+bn1)	32 → 28	960	840	884,736	774,144
conv2 (+bn2)	64 → 64	18,624	16,320	4,718,592	4,128,768
conv3 (+bn3)	128 → 94	74,112	54,426	4,718,592	3,465,216
fc1	256 (inputs 2048 → 1504)	524,544	385,280	524,288	385,024
fc2	10	2,570	2,570	2,560	2,560
Total	–	**620,810**	**459,436**	**10,848,768**	**8,755,712**

Source: widths and totals from the notebook; per-layer values computed from Equations (1) and (2). Parameters include batch-norm weights and biases.

Most of the saving comes from the classifier. Of the 161,374 parameters removed, fc1 accounts for 139,264 (86.30%), conv3 for 19,686 (12.20%), conv2 for 2,304 (1.43%) and conv1 for 120 (0.07%) (*analysis*). Pruning conv3 channels shrinks fc1 because each removed channel takes 16 inputs with it. Right after pruning, and before any retraining, the student scored only 61.64% on the validation set, 17.80 points below the teacher's 79.44% (Notebook).

6.3.3 Knowledge Distillation Retraining

Table 6.7 lists the retraining log of the pruned student (Notebook). Validation accuracy rose from 75.66% after the first epoch to a best of 78.98% at epoch 8. Epochs took about 12.5 s on average (99.8 s in total, *analysis* from the log). The loss shown is the combined KD loss, so it is not comparable with the cross-entropy values in Table 6.4.

Table 6.7: Retraining log of the pruned student with KD (Notebook)

Epoch	Training loss (combined)	Training accuracy (%)	Validation accuracy (%)	Time (s)
1	0.3166	86.17	75.66	13.6
2	0.2357	88.04	78.04	12.4
3	0.2124	88.95	77.42	12.4

Epoch	Training loss (combined)	Training accuracy (%)	Validation accuracy (%)	Time (s)
4	0.2063	89.51	78.44	12.4
5	0.2113	89.63	77.22	12.1
6	0.2061	89.97	78.62	12.2
7	0.1943	90.25	78.00	12.3
8	**0.1955**	**90.74**	**78.98**	**12.4**

Source: output of the KD training cell of the notebook. Best validation accuracy: 78.98% (epoch 8).

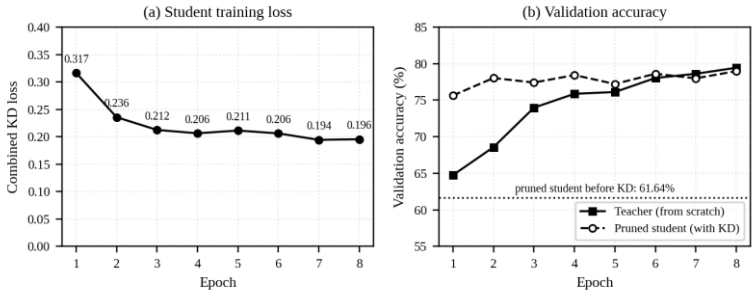


Figure 6.4: Student training loss and validation accuracy during KD retraining, compared with the teacher. *Source: values from Table 6.4 and Table 6.7 (Notebook).*

6.3.4 Final Test-Set Comparison

The best teacher and student checkpoints were each evaluated once on the 10,000 test images. Table 6.8 gives the final comparison and Figure 6.5 plots it.

Table 6.8: Final comparison of the teacher CNN (M1) and the GA-pruned student with KD (M5) (Notebook)

Metric	Teacher CNN (M1)	GA-pruned CNN + KD (M5)	Change
Test accuracy (%)	79.33	79.00	−0.33 points
Macro precision	0.7924	0.7905	−0.0019
Macro recall	0.7933	0.7900	−0.0033
Macro F1-score	0.7927	0.7899	−0.0028
Parameters	620,810	459,436	−25.99%
MACs (millions)	10.8488	8.7557	−19.29%
FLOPs (millions, 2 × MACs)	21.6975	17.5114	−19.29%
Latency (ms per image)	1.3488	1.6645	+23.41% (slower)
Saved weight file (KB)	2,435.1	1,804.9	−25.88% (derived)

Source: notebook output; the file-size change is derived from the two printed sizes.

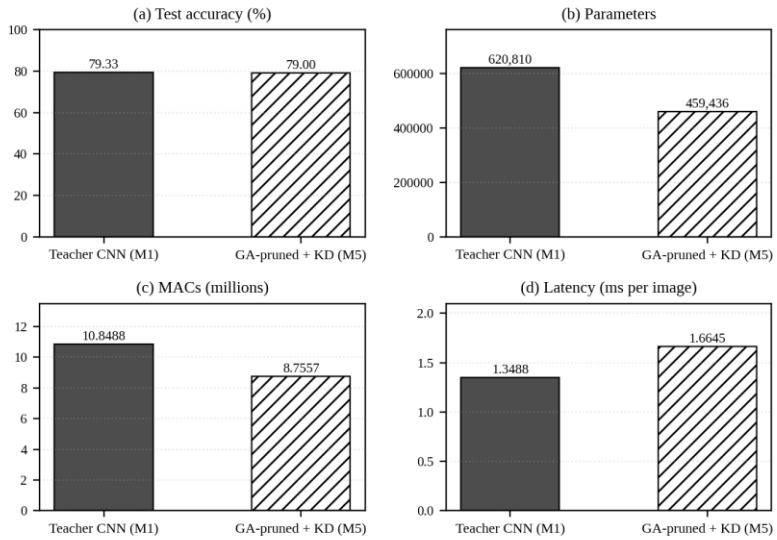


Figure 6.5: Teacher and pruned student compared on test accuracy, parameters, MACs and latency.
Source: values from Table 6.8 (Notebook).

The student keeps almost all of the teacher's accuracy (0.33 points lower, about 0.42% in relative terms) while using 25.99% fewer parameters and 19.29% fewer MACs. Its single-image latency, however, is 23.41% higher.

6.3.5 Per-Class Performance of the Student

Table 6.9 gives the student's per-class results on the test set, and Figure 6.6 shows its confusion matrix. Automobile (F1 0.89), ship (0.88) and truck (0.86) are the easiest classes, and cat (0.62) is the hardest. The largest confusions are between cat and dog: 150 cats were predicted as dogs and 139 dogs as cats. Automobiles and trucks are also confused with each other (57 and 55 images) (Notebook).

Table 6.9: Per-class precision, recall and F1-score of the pruned student with KD on the test set (Notebook)

Class	Precision	Recall	F1-score
airplane	0.81	0.82	0.81
automobile	0.90	0.88	0.89
bird	0.70	0.73	0.71
cat	0.63	0.60	0.62
deer	0.76	0.75	0.75
dog	0.71	0.72	0.71
frog	0.80	0.87	0.83
horse	0.85	0.80	0.83
ship	0.87	0.89	0.88
truck	0.88	0.84	0.86

Source: classification report printed by the notebook (1,000 test images per class).

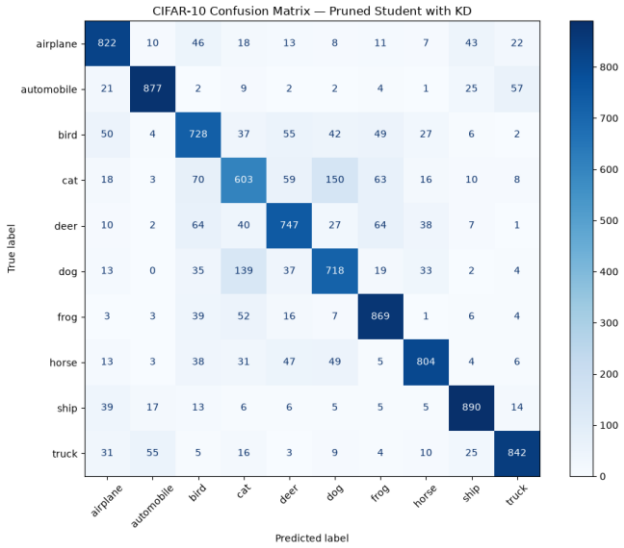


Figure 6.6: Confusion matrix of the pruned student with KD on the CIFAR-10 test set. *Source: figure produced by the student's notebook.*

6.3.6 Status of the Five-Model Comparison

Table 6.10: Measured results and status of the planned comparison (Notebook)

ID	Configuration	Val acc (%)	Test acc (%)	Macro-F1	Params	MACs	Latency (ms)	Status
M1	Original CNN (teacher)	79.44	79.33	0.7927	620,810	10.8488 M	1.3488	Measured
M5	GA pruning + KD (RKD)	78.98	79.00	0.7899	459,436	8.7557 M	1.6645	Measured
M5a	GA-pruned student before retraining	61.64	Not evaluated	Not evaluated	459,436	8.7557 M	Not measured	Validation only
M2	Uniform pruning, no KD	–	–	–	–	–	–	Not run
M3	KD only, small student (no pruning)	–	–	–	–	–	–	Not run
M4	GA pruning, label-loss retraining	–	–	–	–	–	–	Not run

Val acc = validation accuracy on 5,000 images. A dash means the model was not trained in the notebook, so no value exists.

Without M2, M3 and M4, the comparison cannot say how much of the result is due to the GA, to the Taylor scores, or to distillation itself. These runs are listed as future work in Chapter 7.

6.4 Theoretical Interpretation of Results

6.4.1 Percentage Reduction versus Percentage-Point Change

A **percentage reduction** compares a resource before and after pruning, relative to the original value. A **percentage-point change** is the plain difference between two percentages, such as two accuracies:

$$Reduction\ (\%) = \frac{X_{orig} - X_{pruned}}{X_{orig}} \times 100 \quad (21)$$

$$\Delta_{pp} = Acc_{orig} - Acc_{pruned} \quad (22)$$

For classifier D, Equation (22) gives 92.24 − 90.42 = 1.82 percentage points. In relative terms this is 1.82 / 92.24 ≈ 1.97% of the original accuracy (*analysis*), so the abstract's "1.82% decrease" is a percentage-point figure. Equation (21) gives a time reduction of (3.6 − 2.1) / 3.6 = 41.67%, a speed-up of about 1.71×. The pruned classifier keeps about 9% of the original parameters (2.81 / 31.34 ≈ 8.97%). Recomputing from the rounded table entries reproduces the reported percentages to within about 0.1

point, except classifier A's |Param| (recomputed 90.60% against 90.22% reported), because 0.11 M is a heavily rounded value.

Worked example for this study: Equation (22) gives 79.33 − 79.00 = 0.33 percentage points, about 0.42% of the baseline accuracy. Equation (21) gives a parameter reduction of (620,810 − 459,436) / 620,810 = 25.99% and a MAC reduction of (10.8488 − 8.7557) / 10.8488 = 19.29%. For latency the "reduction" is negative: (1.3488 − 1.6645) / 1.3488 = −23.41%, which means the student is 23.41% slower (about 1.23 times the teacher's time).

6.4.2 Accuracy–Efficiency Trade-off

In the paper, classifier D lost 1.82 points for a 91% parameter cut, and the smaller classifiers lost much more (7.97 points for A). In this study, a 25.99% parameter cut cost 0.33 points on the test set. The two are not directly comparable, because the networks, datasets and compression levels differ. Within the experiment, the cheap part of the trade-off was the classifier: 86.30% of the removed parameters came from fc1, which lost inputs when conv3 channels were pruned, and conv2 kept all its channels because the GA set its ratio to zero. A much larger compression, as in the paper, would need conv2 and the fc1 units to be pruned as well (*analysis*).

6.4.3 Fewer FLOPs Do Not Guarantee Proportional Speed

In the paper, FLOPs fell 90.96% for classifier D but time fell only 41.67%. The experiment here shows the same effect in an extreme form: MACs fell 19.29%, yet measured latency rose 23.41%, from 1.3488 ms to 1.6645 ms. Plausible reasons are that a network this small takes about a millisecond on a T4, so the time is dominated by kernel-launch and memory overheads, not arithmetic; that the new widths of 28 and 94 channels are not multiples of 8, which float16 GPU kernels usually prefer; and that one timing run was made. These reasons were not tested (*analysis*). Latency should be repeated on a CPU with fixed threads and on the target device, and the ratios should be chosen so that channel counts stay hardware-friendly.

6.4.4 Observations on the Experiment

The teacher's training accuracy (91.19%) exceeds its validation accuracy (79.44%) by 11.75 points, and the student shows a similar gap (90.74% against 78.98%), so both models over-fit; no data augmentation was used (*analysis*). Eight epochs of KD recovered 17.34 points of validation accuracy, leaving the student 0.46 points below the teacher on validation and 0.33 points below on the test set. Because no label-only retraining run exists, how much of this recovery is due to distillation, as opposed to retraining alone, is not known. The GA's own estimate also looks optimistic: the winning fitness of 0.7335 implies about 70.8% accuracy for the channel-masked teacher on the 1,024-image subset (0.7335 − 0.15 × 0.1696, derived), but the physically pruned network scored 61.64% on the full validation set. A likely cause is that masking a channel after the convolution, before batch normalisation, is not the same as removing it (*analysis*, not tested).

6.5 Critical Evaluation

Strengths. The method targets structured pruning, so savings can show up as real speed-up on ordinary hardware. The chromosome is compact (one gene per layer), which keeps the GA tractable for models with tens of millions of parameters. The fitness function weighs accuracy, size and computation together, and RKD reached comparable accuracy in 25% fewer epochs than LRW (Paper).

Cost and dependence. Each candidate needs a fitness evaluation, so the search costs roughly population size × generations evaluations (*analysis*). The paper reports 10 to 20 generations but not the population size, the search time, or whether candidates are retrained before scoring. RKD needs up to 150 epochs of retraining and a pass through the teacher at each step, and the student inherits the teacher's mistakes (*analysis*). Results cover three networks on three datasets; other tasks and architectures are untested.

Inconsistencies and comparison limits. The text of Sec. 4.1 of [1] says the pruned model keeps more than 90.00% accuracy, but Table 1 shows 77.12% and 89.01% for classifiers A and B. The comparison

with [15] and [16] uses different original accuracies (for example 92.89% against 92.63% for ResNet-32 on CIFAR-10) and does not report their parameter reductions, so it is not fully controlled. No standard deviations or repeated runs are reported.

Missing ablation. Table 6.3 compares RKD with LRW, which differ in both the loss and the learning-rate schedule. The paper does not compare GA-found ratios with uniform or random ratios, so the benefit of the GA itself, and the separate effect of distillation, are not isolated (*analysis*). The comparison planned in this study (M2 to M4) would answer this, but it has not been run.

This study's implementation. The GA used here is much smaller than the paper's: six individuals, three generations and three genes, with fitness measured on 1,024 images. It used a simple mean crossover and a fitness that rewards channel reduction, not the paper's parameter and FLOP terms. Distillation used a constant temperature of 4 for 8 epochs, whereas the paper lowers the temperature from 20 to 1 over 150 epochs. All results come from a single run with one split seed, so differences of a few tenths of a point cannot be called significant, and latency was measured once on a GPU with random input.

Reproducibility. Reproducing the work is hard because the paper omits population size, selection and mutation settings, the FLOP-counting convention, the log base and accuracy scale in the fitness, training details for R(2+1)D-18, and data splits (Table 4.2 and Section 3.1.1).

6.6 Applications, Advantages and Limitations

A smaller, faster CNN is useful wherever compute, memory or power is limited: mobile vision, embedded systems, AR/VR gesture recognition (the setting evaluated in [1]), edge AI cameras and sensors, and robots or drones. Only the gesture and CIFAR experiments were run in [1]; the method has not been validated for every application listed here.

Table 6.11: Advantages and limitations of the hybrid approach

Advantages	Limitations
Structured pruning gives real speed-up on standard hardware (Paper)	GA search needs many fitness evaluations; its cost is not reported (*analysis*)
More than 90% fewer parameters and FLOPs for the gesture model, with a 1.82-point drop for classifier D (Paper)	Accuracy loss can be large for small models (7.97 points for classifier A) (Paper, Table 1)
Compact chromosome (one gene per layer) scales to large models (Paper)	Several settings are unspecified, which limits reproducibility (Table 4.2)
RKD reached comparable accuracy in 150 instead of 200 epochs (Paper)	Needs a trained teacher and a long retraining stage (*analysis*)
Balances accuracy, size and computation in one fitness (Paper)	Fewer FLOPs do not guarantee proportional speed (Paper, Table 1)

6.7 Summary of Findings

- The reference paper reports, for classifier D of its gesture model, a 91.02% parameter reduction and a 90.96% FLOP reduction for a 1.82 percentage-point accuracy drop, and a fall in inference time from 3.6 ms to 2.1 ms (Paper).
- RKD reached accuracy close to LRW in 150 instead of 200 epochs on ResNets (Paper).
- The student's baseline CNN has 620,810 parameters and 10.8488 M MACs, and scored 79.33% on the test set with a macro-F1 of 0.7927 and a latency of 1.3488 ms per image (Notebook).
- After Taylor-guided, GA-selected pruning (28, 64 and 94 channels) and KD retraining, the student has 459,436 parameters (−25.99%) and 8.7557 M MACs (−19.29%), and scores 79.00% with a macro-F1 of 0.7899 (−0.33 points) (Notebook).

- KD recovered the pruned student from 61.64% to 78.98% validation accuracy, but measured latency rose by 23.41%, so fewer operations did not mean faster inference here (Notebook).
- Models M2 to M4 were not run, so the separate benefit of the GA and of KD is not established.

7. Conclusion and Future Scope

7.1 Conclusion

Deep CNNs are accurate but costly to store and run, which makes them hard to deploy on constrained devices. The reference paper [1] proposes a hybrid solution: a Genetic Algorithm searches one pruning ratio per layer, the chosen channels are removed, and the pruned network is retrained with Knowledge Distillation from the original model. The evolutionary search replaces manual tuning of layer-wise ratios with a population-based search guided by a fitness that weighs accuracy, size and operations, and distillation lets the pruned network learn from the teacher's soft outputs with a temperature that falls from 20 to 1. The paper reports that its largest gesture classifier lost 91.02% of its parameters and 90.96% of its FLOPs for a 1.82 percentage-point accuracy drop, with inference time falling from 3.6 ms to 2.1 ms.

This study implemented a simplified version of the pipeline on CIFAR-10. A baseline CNN with 620,810 parameters reached 79.33% test accuracy. Taylor-guided, GA-selected pruning left 28, 64 and 94 channels, and after eight epochs of KD the student had 25.99% fewer parameters and 19.29% fewer MACs, with 79.00% test accuracy (0.33 points lower) and a macro-F1 of 0.7899. KD recovered the student from 61.64% to 78.98% validation accuracy. Measured single-image latency, however, rose from 1.3488 ms to 1.6645 ms, which supports the paper's own caution that fewer operations do not guarantee faster inference.

7.2 Limitations

- The GA is a reduced version (6 individuals, 3 generations, 3 genes) and its fitness uses a masked-teacher accuracy on 1,024 images, which proved optimistic compared with the physically pruned network.
- All results come from a single run, so the 0.33-point accuracy difference has no error bars.
- Models M2 to M4 were not run, so the separate benefits of the GA, of Taylor scoring and of distillation are not isolated.
- Latency was measured only on a Tesla T4 GPU with float16 and a single image; CPU and edge devices were not tested, and latency got worse.
- The compression (25.99% fewer parameters) is modest compared with the paper's 91%, because the baseline is small, only three layers were pruned, and ratios were capped at 0.5. The baseline over-fits and was trained for eight epochs without augmentation.

7.3 Roadmap and Future Scope

Table 7.1: Next steps for completing the experiment

Step	Work	Status
1	Train the baseline CNN (teacher) and save the best checkpoint	Done (Notebook)
2	Compute Taylor channel-importance scores	Done (Notebook)
3	Run a GA search for layer-wise pruning ratios	Done, simplified (Notebook)
4	Prune physically, retrain with KD, evaluate on the test set and measure MACs and latency	Done (Notebook)

Step	Work	Status
5	Run M2 (uniform pruning, no KD), M3 (KD only, small student) and M4 (GA pruning, label-loss retraining)	Not run
6	Repeat with 3 to 5 seeds; measure latency on CPU and edge devices; use the paper's fitness and temperature schedule	Not run

Beyond this plan, future work could add the paper's fitness (with parameter and FLOP terms) and a larger population, lower the distillation temperature during retraining as the paper does, prune conv2 and the fc1 units as well, keep channel counts at multiples of 8 for GPU efficiency, and combine pruning with quantisation.

8. Appendix I: Notebook Summary, Glossary and Verification

8.1 Notebook Cells and Status

Cell	Purpose	Result printed (Notebook)
1	Upgrade torch, torchvision, scikit-learn, matplotlib, pandas	torch 2.14.1, torchvision 0.29.1, scikit-learn 1.9.1, matplotlib 3.11.2, pandas 3.1.0; dependency warnings for google-colab and cudf
2–4	Check PyTorch and GPU; load and split CIFAR-10	CUDA available; Tesla T4, 15.64 GB; 45,000 / 5,000 / 10,000 images
5–6	Select device; build the CNN	cuda; 620,810 parameters
7	Train the teacher for 8 epochs	Best validation accuracy 79.44%
8	Taylor channel importance	Score ranges for conv1, conv2, conv3
9	GA search	Best ratios [0.095, 0, 0.26]; fitness 0.7335
10–11	Physical pruning; accuracy before retraining	28 / 64 / 94 channels; 459,436 parameters (−25.99%); validation accuracy 61.64%
12	KD retraining for 8 epochs	Best validation accuracy 78.98%
13–14	Test metrics; confusion matrix; training plots	Teacher 79.33%, student 79.00%; macro-F1 0.7927 and 0.7899
15–17	Latency, MACs, final table and saved files	Latency 1.3488 and 1.6645 ms; MACs 10.849 M and 8.756 M; files 2,435.1 KB and 1,804.9 KB
18–26	Charts, summaries, CSV files and downloads	Comparison charts and CSV files saved

8.2 Glossary

Term	Meaning
Channel / filter	One feature map of a convolutional layer / the weights that produce it
Structured pruning	Removing whole channels or filters, so the layer becomes smaller
Unstructured pruning	Setting individual weights to zero, leaving the tensor shape unchanged
Chromosome / gene	A candidate solution / one element of it (here, one layer's pruning ratio)
Fitness	Score that tells the GA how good a candidate is
Crossover / mutation	Mixing two parents / randomly changing genes
Teacher / student	The original, larger model / the smaller pruned model that is trained
Logit	Raw output of the last layer before softmax
Temperature (T)	Value that softens (T > 1) the softmax output

Term	Meaning
KL divergence	Measure of how different two probability distributions are
FLOP / MAC	Floating-point operation / multiply–accumulate operation
RKD	Retraining with Knowledge Distillation, the retraining method of [1]
LRW / FT	Learning rate rewinding / fine-tuning, label-only retraining methods [6]

8.3 Numerical Verification Log

Item checked	Location in [1]	Result
Classifier D: 92.24 → 90.42 %, drop 1.82	Table 1; Sec. 4.1; abstract	Matches
Classifier D: 31.34 → 2.81 M (91.02%); 40.52 → 3.66 G (90.96%)	Table 1; abstract	Matches
Classifier D: 3.6 → 2.1 ms	Table 1	Matches (41.67%)
Classifiers A–C values	Table 1	Matches; drops re-computed
ResNet results and RKD vs LRW	Tables 2 and 3	Matches; drops re-computed
κ, λ, α, β, γ, δ, T_0, E, noise range, generations	Sec. 3.1, 4	Matches
“More than 90.00% accuracy after pruning”	Sec. 4.1	Inconsistent with Table 1 (A: 77.12%, B: 89.01%)
Third fitness coefficient printed as “r”	Sec. 3.1.4, EQ 10	Read as γ (typographical inconsistency)
Baseline parameter total 620,810	Notebook (printed)	Layer sums in Table 5.1 add up
Parameter reduction 25.99%	Notebook, cells 10 and 16	(620,810 − 459,436) / 620,810 = 25.99%; matches
MAC reduction 19.29%	Notebook, cell 17	(10.8488 − 8.7557) / 10.8488 = 19.29%; matches
Pruned parameters from widths 28 / 64 / 94	Recomputed	459,436; matches the notebook
Pruned MACs from widths 28 / 64 / 94	Recomputed	8,755,712 (8.7557 M); matches the notebook
Teacher MACs from layer shapes	Recomputed	10,848,768 (10.8488 M); matches the notebook
Accuracy difference 79.33 → 79.00	Notebook, cells 13 and 21	−0.33 points; matches. Typed values in later cells equal the computed test results
Latency change 1.3488 → 1.6645 ms	Notebook, cell 16	+23.41% (slower); matches

9. References

[1] K.-L. Chou, C.-L. Wang, Y.-C. Chen, W. Tang, Y.-T. Li, S.-C. Chang, and C.-Y. Wang, "CNN model optimization using a hybrid approach of genetic algorithm-based pruning and retraining with knowledge distillation," in *Proc. Great Lakes Symp. VLSI (GLSVLSI '25)*, New Orleans, LA, USA, Jun. 30–Jul. 2, 2025, pp. 667–672, doi: 10.1145/3716368.3735193.

[2] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[3] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Proc. NeurIPS*, 2015.

[4] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. ICCV*, 2017, pp. 2736–2744.

[5] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," arXiv:1611.06440, 2016.

[6] A. Renda, J. Frankle, and M. Carbin, "Comparing rewinding and fine-tuning in neural network pruning," in *Proc. ICLR*, 2020.

[7] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[8] G. Syswerda, "Uniform crossover in genetic algorithms," in *Proc. Int. Conf. Genetic Algorithms (ICGA)*, vol. 3, 1989.

[9] O. Köksoy and T. Yalcinoz, "Robust design using Pareto type optimization: A genetic algorithm with arithmetic crossover," *Comput. Ind. Eng.*, vol. 55, no. 1, pp. 208–218, 2008.

[10] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016, pp. 770–778.

[12] D. Tran, H. Wang, L. Torresani, J. Ray, Y. LeCun, and M. Paluri, "A closer look at spatiotemporal convolutions for action recognition," in *Proc. CVPR*, 2018, pp. 6450–6459.

[13] Y. Zhang, C. Cao, J. Cheng, and H. Lu, "EgoGesture: A new dataset and benchmark for egocentric hand gesture recognition," *IEEE Trans. Multimedia*, vol. 20, no. 5, pp. 1038–1050, 2018.

[14] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.

[15] H. Cheng, Z. Wang, L. Ma, X. Liu, and Z. Wei, "Multi-task pruning via filter index sharing: A many-objective optimization approach," *Cogn. Comput.*, vol. 13, no. 4, pp. 1070–1084, 2021.

[16] H. Cheng, Z. Wang, L. Ma, Z. Wei, F. E. Alsaadi, and X. Liu, "Differentiable channel pruning guided via attention mechanism: A novel neural network pruning approach," *Complex Intell. Syst.*, vol. 9, no. 5, pp. 5611–5624, 2023.

[17] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.

Reference [1] is the primary source and was read in full for this report. References [2]–[16] are works cited by [1] and are listed as secondary sources for background; [17] is cited for the implementation. Venue details of [11] and [12] follow the original publications.